

Managing API documentation

Plans supporting this feature: **Professional** **Business** **Enterprise**

The API documentation feature in Document360 makes it easy to create clear, interactive documentation by uploading your API specification files. This process automatically builds detailed documentation that covers everything from API endpoints to methods and responses, helping developers understand and use your API more effectively.

Generating API documentation

There are three methods to generate API documentation in Document360:

- From a URL
- From a JSON/YAML/YML file
- With a CI/CD flow

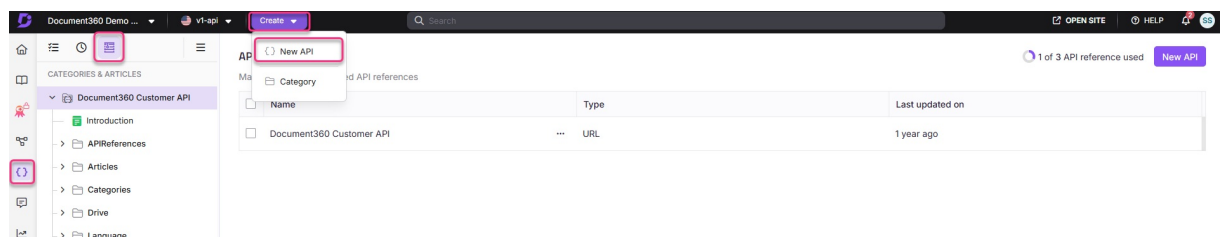
ⓘ NOTE

Document360 supports OpenAPI 2.0, OpenAPI 3.0, OpenAPI 3.1, and Postman API specifications.

Generating API documentation from an API specification file

To generate API references from an API specification file (JSON/YAML/YML),

1. Navigate to the desired project in the Knowledge base portal.
2. Select **API documentation ({ })** from the left navigation bar.
3. Click the **Create** dropdown in the top navigation bar and select **New API**. This will display the **New API reference** window.



4. Select the **Upload API definition** radio button in the **New API reference** window.

5. Click either **Upload from my device** or **Choose from Drive** to select the file from your device or Document360 Drive, respectively. You can also drag and drop your file directly into the New API reference window.

Learn more'. Below this, there are four radio buttons: 'Upload API definition' (selected), 'Create from URL', 'CI/CD Flow', and 'Try sample pet store API file'. Below the radio buttons is a large dashed box containing a file upload icon (a document with a plus sign) and two buttons: 'Upload from my device' and 'Choose from Drive'. Below these buttons is the text 'or drag and drop'. A red arrow points from the left towards the 'Upload from my device' button. Below the dashed box, it says 'Supported formats: .JSON, .YAML, .YML'. At the bottom right, there are two buttons: 'Cancel' and 'New API reference'."/>

New API reference

Document360 supports OpenAPI (3.1, 3.0, 2.0) and Postman Collections, simplifying your API documentation and management. [Learn more](#)

☒ Upload API definition ☐ Create from URL ☐ CI/CD Flow ☐ Try sample pet store API file

Upload from my device Choose from Drive

or drag and drop

Supported formats: .JSON, .YAML, .YML

Cancel New API reference

❗ NOTE

The API documentation import process supports second-level subcategories using the **>** symbol in OpenAPI tags (e.g., **Pets > Details**). This allows for cleaner, nested organization of endpoints, preserves the structure during resync, and ensures proper display and navigation in the left panel of the API reference.

If your uploaded file has any associated alerts or warnings, you can view them by expanding the **Alerts** and **Warnings** dropdowns, which appear in the New API reference window.

6. Click **New API reference** to navigate to the **Publish confirmation** window. You will see a success message in the window, along with the number of categories and articles that have been created.
7. Click **Publish** to generate your API documentation.

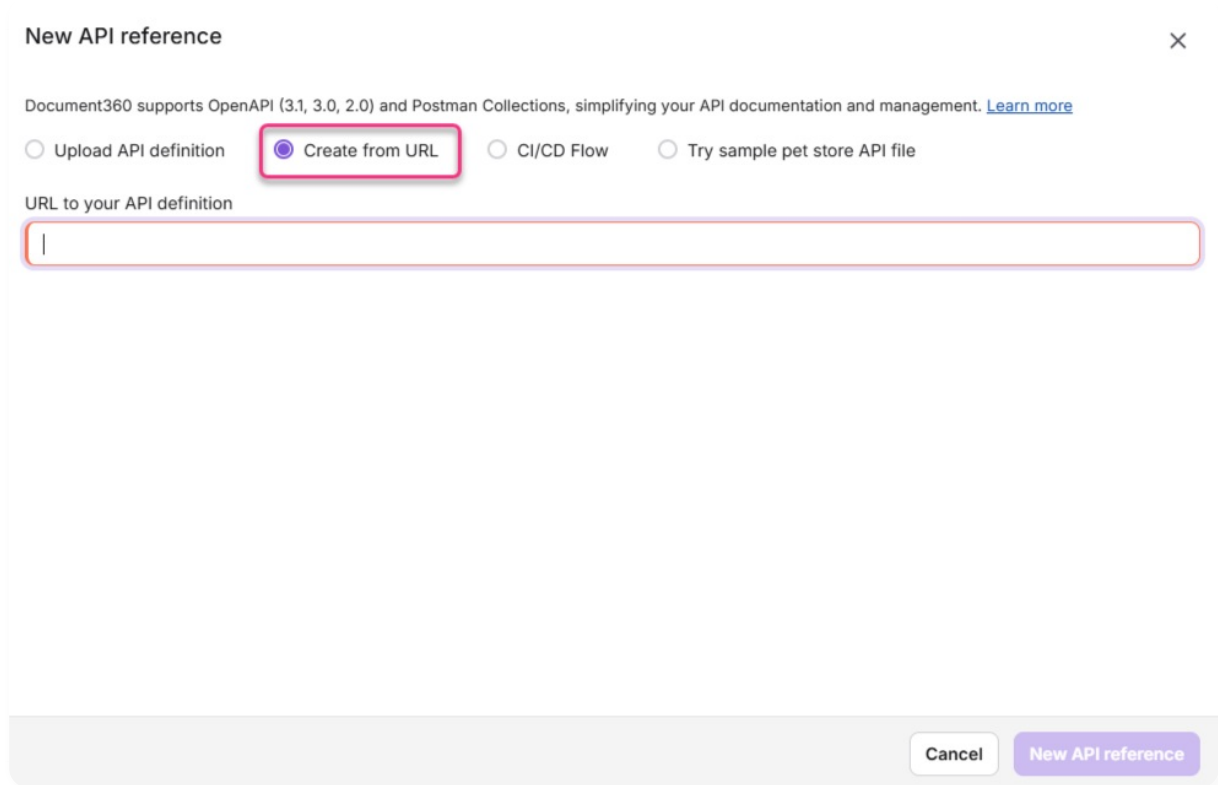
❗ NOTE

To review your documentation before publishing, click **Close** to return to the Documentation screen. Your draft will be visible in the **Categories & Articles** pane.

Generating API documentation from a URL

To upload the API specification file as a URL to Document360,

1. Navigate to the desired project in the Knowledge base portal.
2. Select **API documentation** ({ }) from the left navigation bar.
3. Click the **Create** dropdown from the top navigation bar and select **New API** or click the **New API** button from the top right corner. This will display the **New API reference** panel.
4. In the **Choose source** screen, select the **Create from URL** radio button, and click **Next**.
5. In the **Source settings** screen, enter the URL for your API specification file in the **URL** field.



6. Click **Add API reference** to navigate to the **Finish** screen.
7. On the **Finish** screen, you will be able to see the number of categories and articles that have been created for your API specification file.
8. Click **Publish** to generate your API documentation.

Generating API documentation with a CI/CD flow

Before you upload an API specification file with a CI/CD flow, ensure that the latest version of Node.js is installed on your system. If you are unfamiliar with Node.js, refer to [this guide](#) for installation instructions.

To upload the API specification file with a CI/CD flow,

1. Navigate to the desired project in the Knowledge base portal.
2. Select **API documentation ({ })** from the left navigation bar.
3. Click the **Create** dropdown in the top navigation bar and select **New API**. This will display the **New API reference** window.
4. In the **Choose source** screen, select the **CI/CD Flow** radio button.
5. Copy the full CLI command shown from the **New API reference** window.

New API reference

Document360 supports OpenAPI (3.1, 3.0, 2.0) and Postman Collections, simplifying your API documentation and management. [Learn more](#)

☐ Upload API definition ☐ Create from URL ☒ CI/CD Flow ☐ Try sample pet store API file

1. Ensure you have the latest version of Node.js installed on your system [Learn how to install Node.js](#).

2. Open your command-line interface (CLI) and install the Document360 npm package using the command: (`npm install d360 -g`).

3. Copy the following code into your CLI, replacing placeholders like the file path or URL with your specific values. The API key provided below is linked to your project and is valid only for this project.

npm install d360 -g

d360 apidocs

--userId=

--path=file path / URL

--versionId=

--apiKey=

Cancel

6. In the copied command, update the `--path` value with:

- The full path to your local JSON/YAML/YML spec file. For example,

Bash	Copy
<code>--path=/Users/yourname/projects/api/openapi.yaml</code>	

- A valid URL pointing to your API spec file. For example,

Bash	Copy
<code>--path=https://example.com/api/openapi.yaml</code>	

7. Paste the updated command in your terminal and press Enter.
8. Your API specification file will be uploaded, and API documentation will be generated.

Page: 4 of 21

NOTE

- The first line (`npm install d360 -g`) installs the Document360 CLI tool. You only need to run it the first time. If it's already installed, you can skip that line.
- If you regenerate the API key by clicking the (🔄) icon in the UI, **you must update the `--apiKey` value in your CLI command** before running it again. The old key will no longer be valid.

Managing webhooks (events)

When your OpenAPI 3.1 spec includes webhooks, Document360 imports them and shows an event icon next to each webhook. The page displays the payload schema and example. If the spec does not include an example, a default example and a sample payload are shown. Try it is not available for webhooks. Webhooks are included in resync operations, and your custom content is retained after regeneration.

Regenerate API documentation

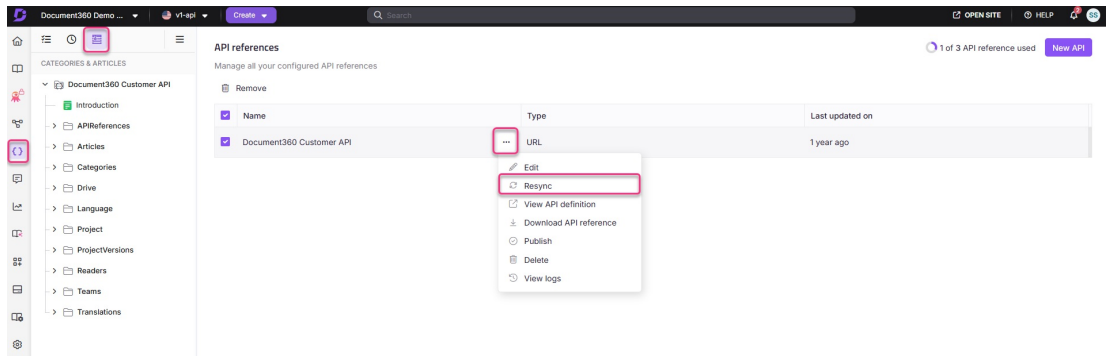
If you make any changes to your API, such as adding new endpoints, you do not need to update your API documentation in Document360 manually. You can regenerate your API documentation, and any changes to your API will be automatically reflected in the updated documentation.

ⓘ NOTE

Any custom content you add to your API documentation will be retained when you regenerate your API documentation.

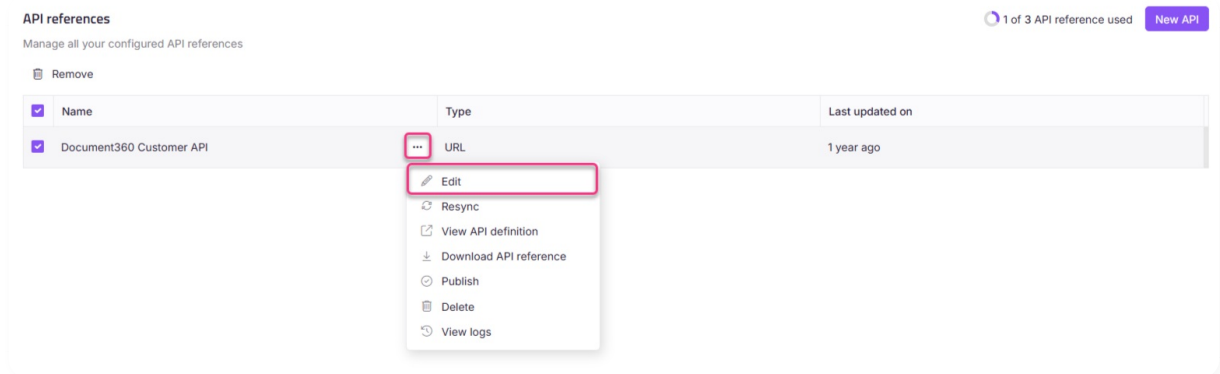
Regenerate API documentation from URL

1. Navigate to the desired project in the Knowledge base portal.
2. Select **API documentation** ({ }) from the left navigation bar.
3. Click **API references** from the left navigation list pane.
4. Click the **More** (⋮) icon next to the desired API reference for which you want to regenerate the API documentation.
 - a. **To regenerate API documentation from the existing URL:**
 - Click **Resync**.
 - The API documentation will be regenerated as per the latest API specification file.



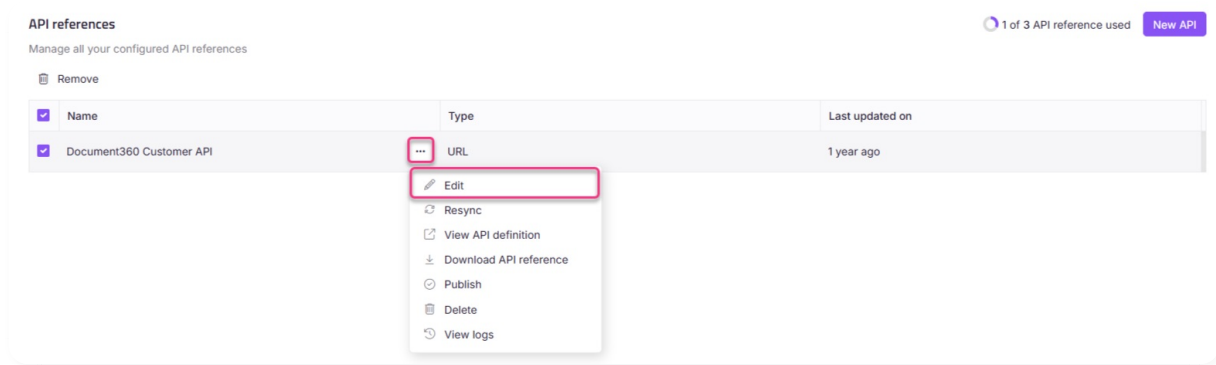
a. To regenerate API documentation with a new URL:

- Click **Edit**.
- Enter the new URL in the **URL** field.
- Click **Update**.
- The API documentation will be regenerated according to the API specification file at the new URL.



Regenerate API documentation from an API specification file

1. Navigate to the desired project in the Knowledge base portal.
2. Select **API documentation ({ })** from the left navigation bar.
3. Click **API references** from the left navigation list pane.
4. Click the **More (⋮)** icon next to the desired API reference for which you want to regenerate the API documentation.
5. Click **Edit**.
6. Upload the latest API specification file in JSON/YAML/YML format.
7. Click **Update**. The API documentation will be regenerated as per the latest API specification file.



Regenerate API documentation integrated with CI/CD flow

You can resync the API reference in your CI/CD pipelines with the help of your [d360 npm packages](#). D360 is the command line tool that helps you set up workflows that sync your API docs to Document360.

You can also perform the resync manually using the command below.

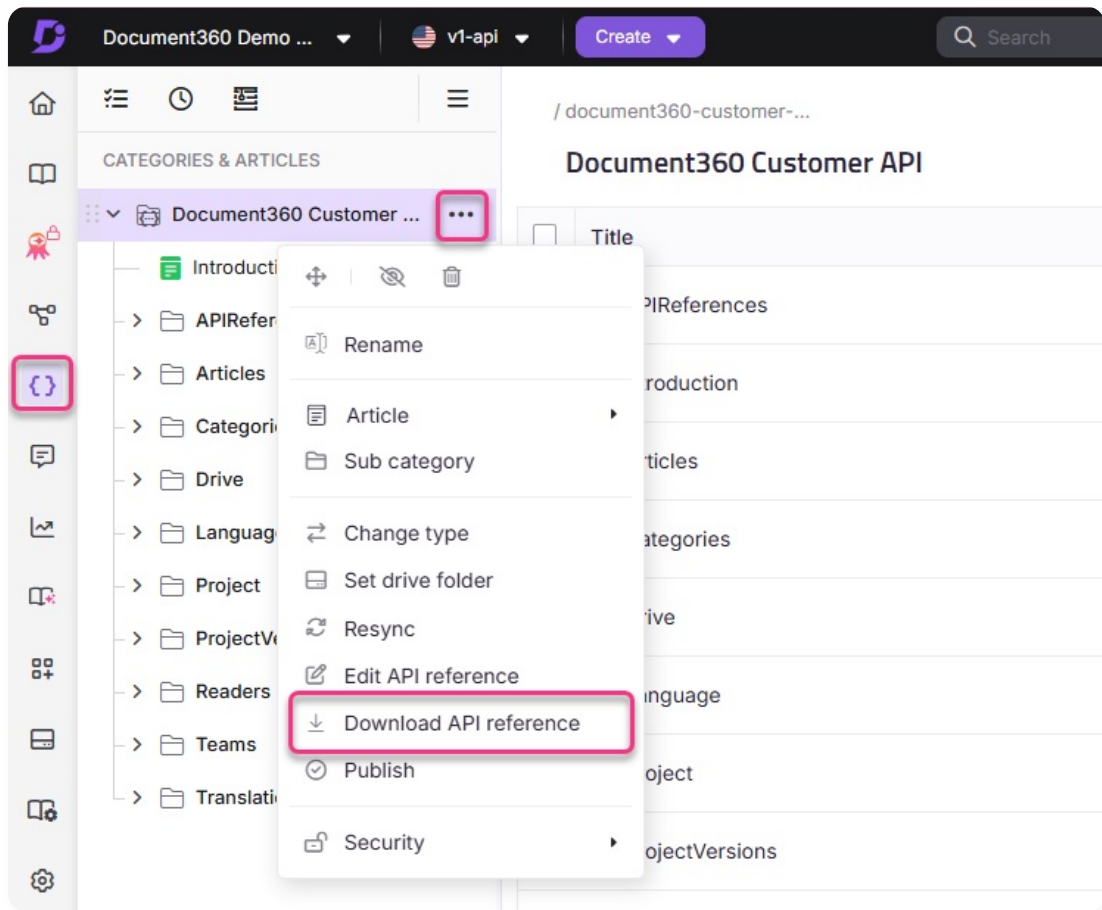
Plaintext	Copy
<pre>d360 apidocs:resync --apiKey=API_key_value --userId=user_id_value --apiReferenceId=API_reference_value --path=Spec_file_path</pre>	

Download API reference

Knowledge base portal

To download your API reference file from the Knowledge base portal, follow the steps below:

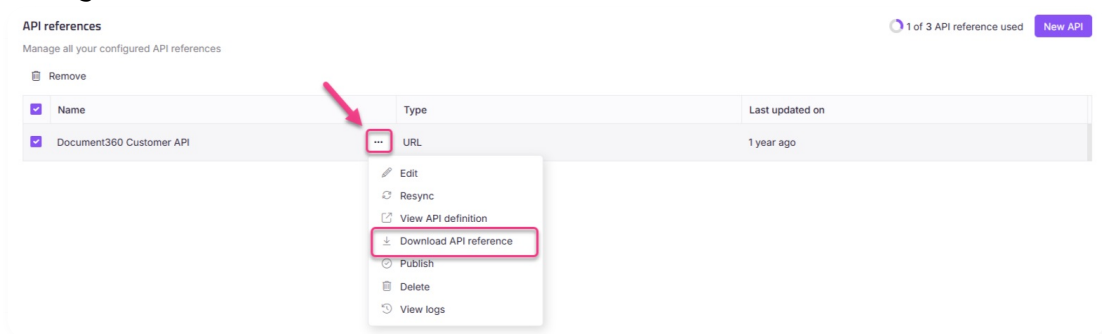
1. Navigate to the desired project in the Knowledge base portal.
2. Click the **More** (⋮) icon next to the desired API reference from the left navigation pane for which you want to download the API reference.
3. Click **Download API reference**.
The latest version of the API reference file will be downloaded to your local storage.



Alternatively,

4. Click **API references** from the left navigation pane.
5. From the configured API references listed, click the **More (⋮)** icon next to the desired API reference for which you want to download the API reference.
6. Click **↓ Download API reference**.

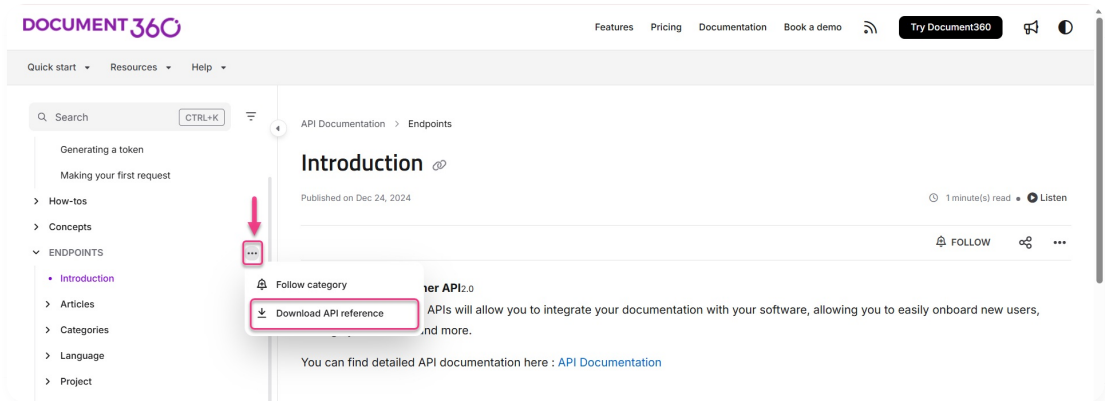
The latest version of the API reference file will be downloaded to your local storage.



Knowledge base site

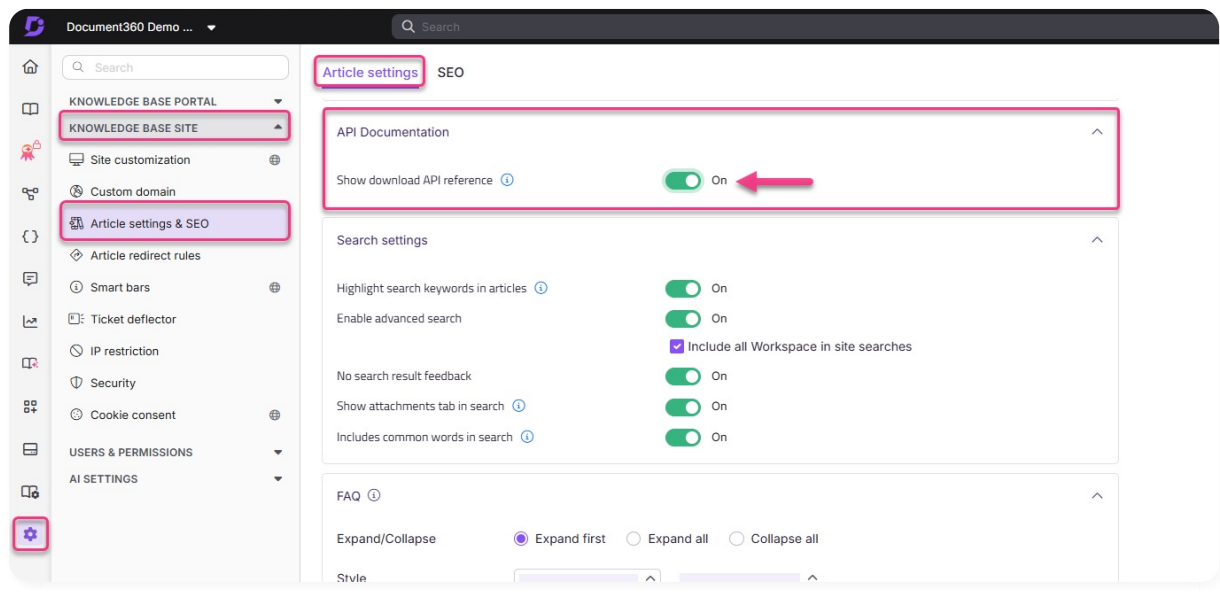
To download your API reference file from the Knowledge base site, follow the steps below:

1. On the API documentation Knowledge base site, from the left navigation pane, click the **More (⋮)** icon next to the desired API reference for which you want to regenerate the API documentation.
2. Click **Download API reference**.
It will be downloaded in a standard format such as .json or .yaml.
3. The Download API Reference option is accessible for all upload types, including:
 - File upload
 - CI/CD flow
 - URL upload



⚠ NOTE

To download the API files, ensure that the **Show download API reference** toggle in the Knowledge base portal settings is turned on.



Troubleshooting

Missing or incorrect server URL in OpenAPI

Error: This error occurs when the **server URL is missing** or incorrectly configured in the OpenAPI specification. As a result, API users do not know where to send API requests, and the "Try it" button may be unavailable in the API documentation. The root cause is typically a missing or incorrect `servers` section in the OpenAPI spec.

Steps to resolve

To resolve this issue, ensure that the OpenAPI specification includes the correct server URL:

1. Define the server URL in the OpenAPI specification:

YAML	Copy
<pre>servers: - url: https://apihub.document360.io description: Main API hub</pre>	

2. For region-based APIs, define multiple server URLs:

YAML	Copy
<pre>servers: - url: https://apihub.document360.io description: Global API hub - url: https://apihub.us.document360.io description: US region API hub</pre>	

3. Ensure that all API clients (e.g., Postman, cURL) use the correct server URL when making requests.

ⓘ NOTE

The above URLs are examples. Please refer to your specific API configuration for the correct server URL.

Example:

YAML example for missing `servers` section:

```
paths:
  /users:
    get:
      summary: Retrieve a list of users
      responses:
        '200':
          description: A list of users
          content:
            application/json:
              schema:
                type: array
                items:
                  type: object
                  properties:
                    userId:
                      type: integer
                    userName:
                      type: string
```

To avoid this issue, ensure the `servers` section is properly defined in the OpenAPI specification to prevent API consumers from having to construct API request URLs manually.

401 unauthorized error from incorrect server URL

Error: This error occurs when API requests return a **401 unauthorized** response. It typically happens when the **incorrect server URL** is used in API requests. For example, using `https://apihub.document360.io` instead of `https://apihub.us.document360.io` causes authentication failures.

Steps to resolve:

To resolve this issue, follow these steps to ensure correct server URL usage and proper authentication:

1. Ensure that API requests use the correct region-based endpoint.
2. Confirm that the correct client ID and client secret are being used.

Example of a correct API request:

HTTP	Copy
GET https://apihub.us.document360.io/v2/Articles/{article_id}/en?appendSASToken=true	

ⓘ NOTE

The above URL is an example. Please refer to your specific API configuration for the correct server URL.

3. Ensure the authentication token is correctly included in the request headers.

Missing server URL in API documentation

Error: This issue occurs when API consumers are unsure **which base URL** to use for their requests. This typically happens when the documentation does not clearly state the server URL, even though it might be correctly configured in the OpenAPI specification.

Steps to resolve:

To prevent confusion and ensure API consumers know the correct base URL:

1. Specify the base URL explicitly in the API documentation.

Examples:

Comparison table: server URL present vs. missing

Scenario	Behavior	Example
Server URL is present	API requests work normally	<code>https://apihub.document360.io/users</code>
Server URL is missing	Clients must manually configure	<code>https://your-api-host.com/users</code>

ⓘ NOTE

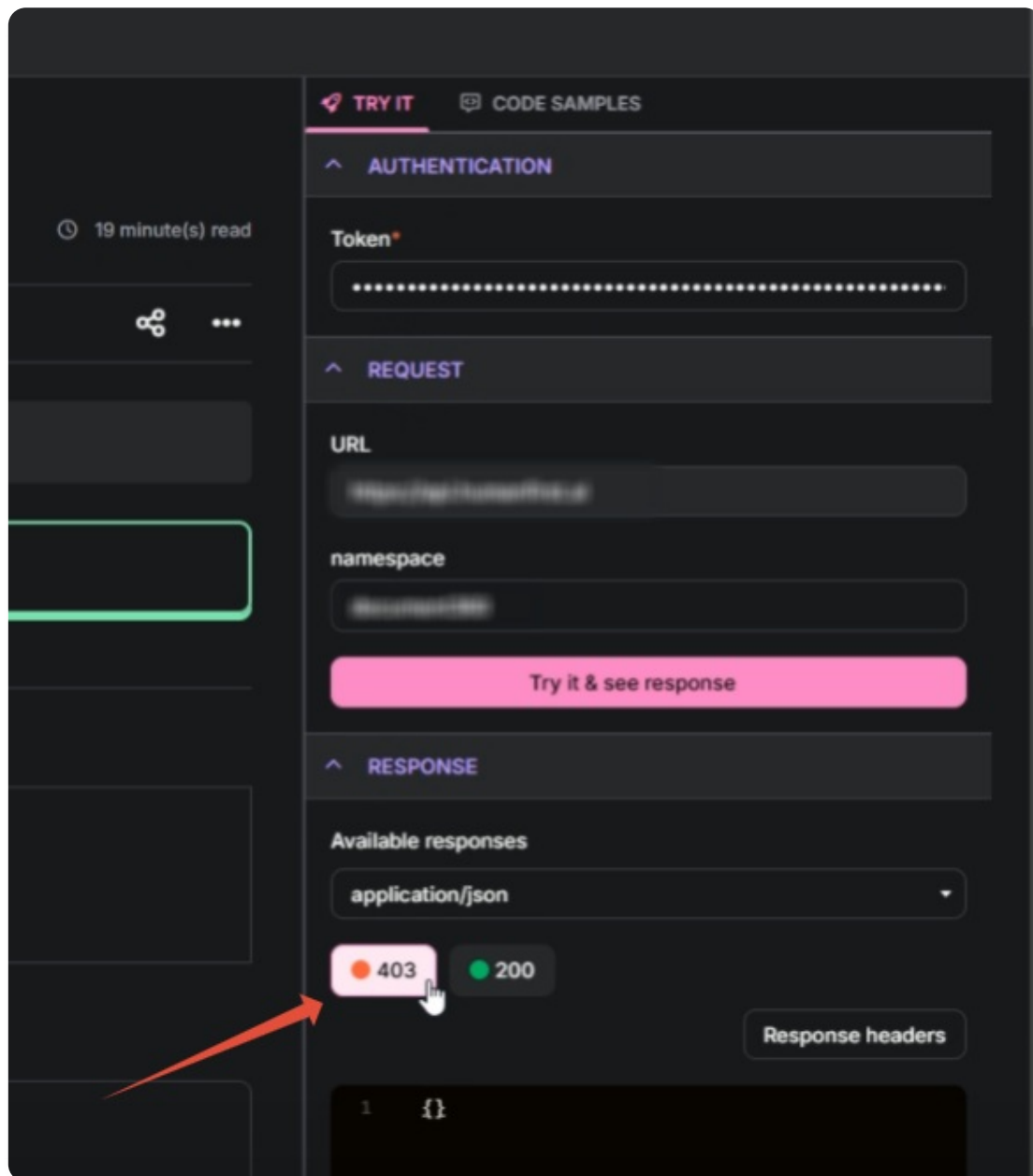
The above URLs are examples. Please refer to your specific API configuration for the correct server URL.

To avoid this issue, ensure the API documentation clearly specifies the server URL to avoid any confusion.

403 error when using the “Try it” feature with an OAS file

Error:

After successfully uploading an OAS file, clicking on the **Try it** feature results in a **403 error**. This error occurs when the OAS file is missing a definition for the **BearerAuth** security mechanism.

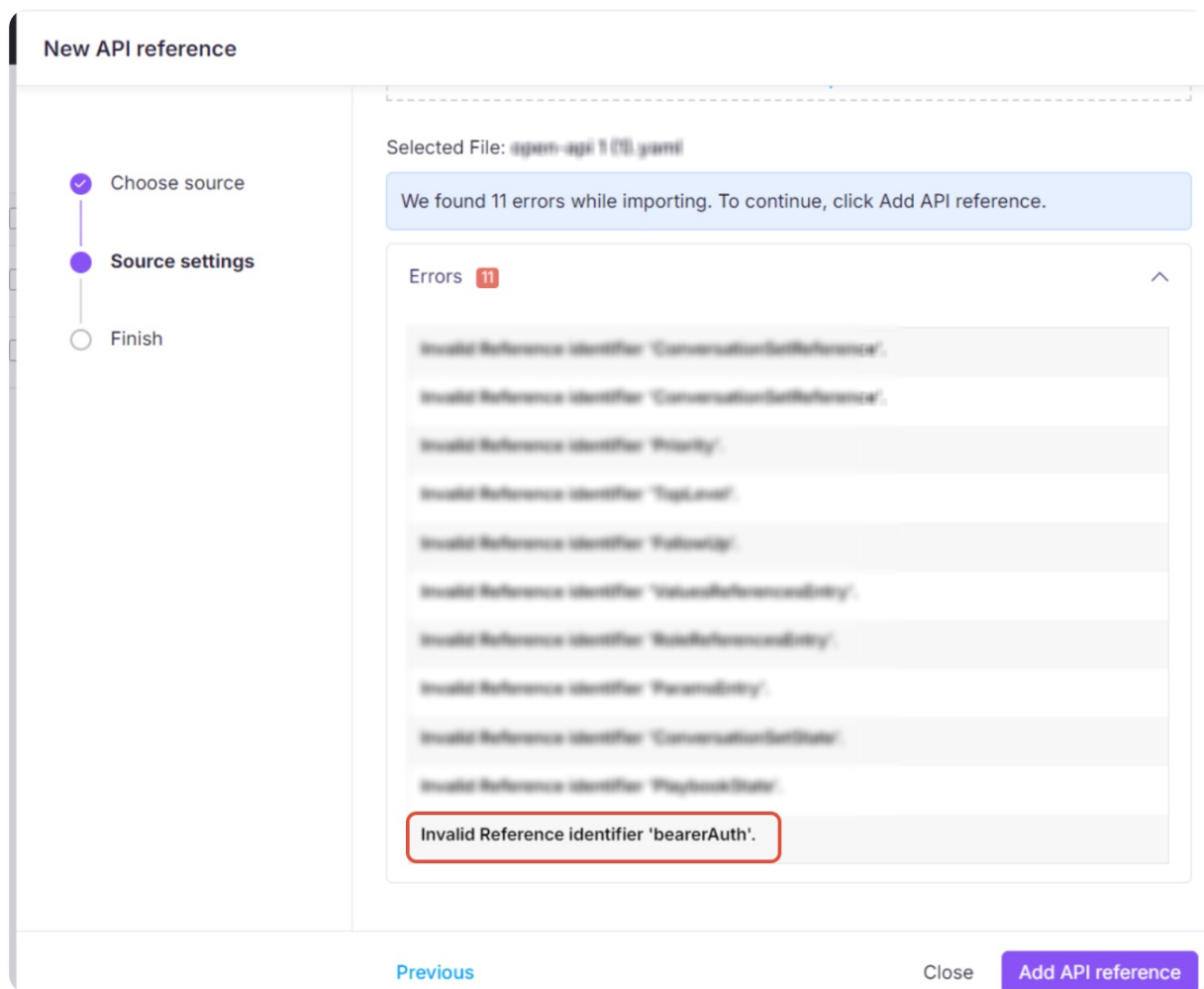


Steps to resolve:

To resolve the 403 error, follow these steps:

1. Check if the **BearerAuth** security definition is present in your OAS file.
2. If the **BearerAuth definition** is missing, include the **BearerAuth** security definition in the OAS file and reupload it.

3. During the import process, verify if any alerts related to **Bearer Authentication** are triggered and address them accordingly.



Incorrect HTML styling in API response

When retrieving an article's HTML content through an API, the response is formatted in **JSON**, which uses **escaping** (`\`) to handle certain characters safely. This escaping alters characters like quotes, backslashes, and newlines to prevent data transmission errors. However, if the escaped characters are not converted back (unescaped), the HTML structure may break, leading to styling issues when rendered.

For example, a simple HTML paragraph:

HTML	Copy
<pre><p>This is a "sample" paragraph.</p></pre>	

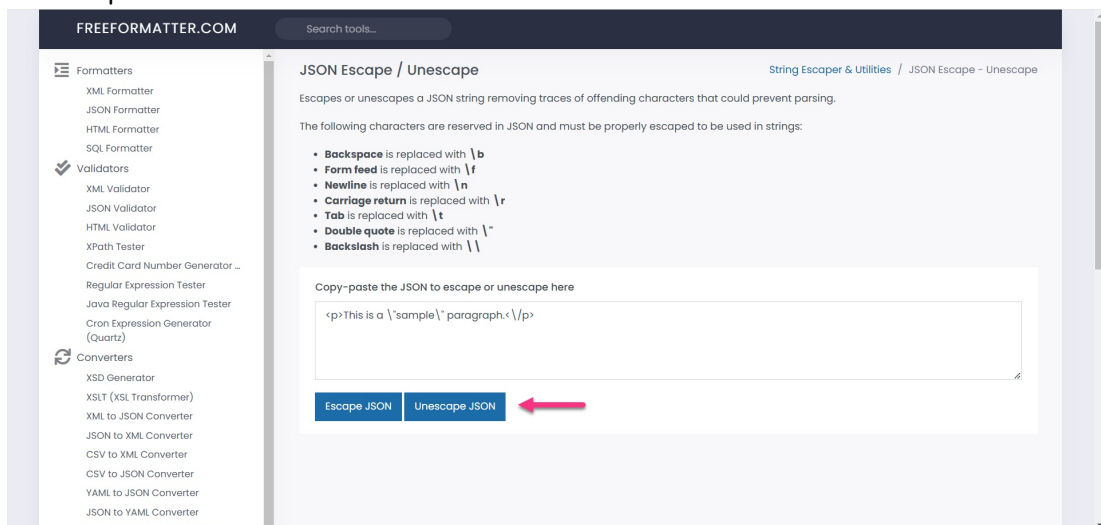
Might appear in the JSON response as:

JSON	Copy
<pre>"<p>This is a \"sample\" paragraph.</p>"</pre>	

The escaped double quotes (`\"`) ensure the JSON remains valid, but if not **unescaped**, the HTML might not render correctly.

Steps to resolve

1. Retrieve the HTML content via the API and inspect the response.
2. Look for escaped characters in the JSON response, such as:
 - `\"` (escaped double quotes)
 - `\\` (escaped backslashes)
 - `\n` (newline)
 - `\t` (tab)
3. **Unescape** the JSON content before rendering it. This will restore the original HTML structure.
4. Verify the styling after unescaping to ensure the content appears correctly.
5. If the issue persists, check if additional modifications in your code are altering the API response.



Once unescaped properly, the HTML will display as expected, preventing styling discrepancies.

Variables and Snippets not rendering in API response

When retrieving article content via the API, variables and snippets may appear **unprocessed** (e.g., `{{variable.UserName}}` instead of actual values). This typically happens when the `"isForDisplay"` parameter is not set correctly.

- If `"isForDisplay" = true` , the API will return the article content with **all variables and snippets fully rendered**. This means the placeholders are replaced with actual values, making the content suitable for display to end-users.
- If `"isForDisplay" = false` (or not set), the API response will contain **unprocessed variables and snippets**, which may not be useful for direct display. If the issue persists after following these steps, please contact the Document360 support team for further assistance: [Contact Document360 Support](#)

Article content in the Knowledge base:

Plaintext	Copy
Welcome, {{variable.UserName}}! Here's how to use {{variable.ProductName}}.	

API response without `"isForDisplay"` :

JSON	Copy
<code>"Welcome, {{variable.UserName}}! Here's how to use {{variable.ProductName}}."</code>	

The variables remain **unprocessed**, making it unsuitable for direct display.

API response with `"isForDisplay"=true` :

JSON	Copy
<code>"Welcome, John! Here's how to use Document360."</code>	

Steps to resolve:

1. Retrieve the article content via API and inspect the response.
2. Check if variables or snippets appear unprocessed in the response.
3. Ensure the API request includes the `"isForDisplay"` parameter. If missing, modify the request to include `"isForDisplay": true` .
4. Send the modified API request. Example request:

JSON	Copy
<pre>{ "articleId": "12345", "isForDisplay": true }</pre>	

5. Verify if the API response now displays the correctly rendered variables and snippets.
6. If the issue persists, ensure that the system processing the API response handles and displays the rendered content properly.
7. When updating articles via the API, do not pass fully rendered content to avoid replacing dynamic placeholders with static values.

Empty body shown in API documentation due to missing schema type

The API documentation displays an empty body. The possible cause for this error could be that the OpenAPI schema is missing the `"type": "object"` attribute in one or more object definitions.

Steps to resolve:

Ensure that every object definition in your OpenAPI specification includes `"type": "object"`. This attribute clearly specifies that the defined structure is an object, helping maintain clarity and consistency in your API documentation. It allows API documentation tools to accurately render request body parameters, response schemas, and other object definitions, making it easier for developers to understand and interact with your API.

Issue in adding an API reference

Error: Unable to add API reference. This operation can't be completed. Please ensure that you have given a valid specification file.

 **Unable to add API reference**
• This operation can't be completed. Please ensure that you have given a valid specification file.

The possible cause might be that the uploaded specification file (YAML) may be malformed and not a valid OpenAPI 3.0 YAML. This often happens when the file is copied or exported from a rich-text editor, introducing formatting issues.

Steps to resolve:

1. **Validate your spec file:** Open your YAML file in a tool like Swagger Editor or a code editor to check for errors or formatting issues.
2. **Look for unwanted formatting characters:** Check for characters like `\f0\fs24` or trailing backslashes `\` that may have been introduced during copy-paste from a rich-text source. These can break the YAML format.
3. **Clean up the file:** Use a plain text or code editor to remove any special formatting characters. Avoid using word processors when editing or saving YAML files.
4. **Re-upload the file:** After cleaning the file and ensuring it's a valid OpenAPI 3.0 YAML, try uploading it again.

If the issue persists after following these steps, please contact the Document360 support team for further assistance: [Contact Document360 Support](#)

FAQ

Can I keep the generated API documentation in the draft state?

After uploading the API reference file, if you click **Close** instead of **Publish**, all the generated API documentation articles are kept in the draft state.

Why does the Try it button URL include tryit.document360.io?

When you click the **Try it & see response** button in API documentation, the request URL will include `tryit.document360.io` prepended to your API's base URL (for example, `https://tryit.document360.io/https://apihub.document360.io/v2/ProjectVersions`). This is the expected behavior. The `tryit.document360.io` subdomain is used internally to route and process API test requests. This does not affect the functionality or the response, requests will still return the correct results as normal.

How do I see the published status for all available languages?

Set the `isPublished` parameter to false when retrieving articles, categories, or documents. This returns every language the article exists in, including languages that only have a draft and have never been published. This gives you a complete view of all languages and their publish states for that content.

The languages listed under Available Languages depend on whether you are filtering for published content or all content using the `isPublished` parameter.

- Published content only (`isPublished` =true): Shows only languages where the article has been published at least once, including languages that have a newer draft on top of an existing published version.
- All content (`isPublished` =false): Shows every language the article exists in, including languages that only have a draft and have never been published.

For example,

Language	Status
French, German, Japanese	Published
Czech	Published, but has a newer draft
Spanish, Portuguese	Draft only — never published

- If `isPublished` = true: Available Languages: French, German, Japanese, Czech
- If `isPublished` = false: Available Languages: French, German, Japanese, Czech, Spanish, Portuguese

This applies when retrieving endpoints, articles, categories, and documents.

Can readers still access Knowledge base site during Document360 portal downtime?

Yes. GET calls of the Customer API now run independently of the Document360 portal, so your readers can continue accessing the site even during scheduled maintenance or portal downtime.

Can I move a specific API endpoint article from one API

reference folder to another in Document360?

No, it is not possible to move a specific API endpoint article from one API reference folder to another. However, you can move API endpoint articles between subfolders within the same API reference folder.

Can an article from the API documentation have the same URL[^] as a Knowledge base article?

No, a knowledge base article and an API documentation article cannot have the same URL. However, you can keep them under the same subdomain.

How often is an API reference file resynced?

If you generated your API documentation from a URL or a JSON, YAML, or YML file, the API reference file is not resynced automatically and will need to be updated manually. To have the API reference file resynced automatically, it is recommended to integrate it with the CI/CD flow.

Why am I getting a 403 error when posting details through the API endpoint?

A 403 error occurs when you attempt to post details through the API endpoint. This happens when the API token used does not have the required permission to make a POST request. Ensure that the API token has the necessary permissions for POST requests. Update the token settings and try again.

How many levels of categories are supported in an API documentation workspace?

The API documentation workspace supports up to three levels of subcategories. If your API specification includes more than three levels, any additional levels will be added and displayed at the third level to maintain a consistent structure.

Can I create nested folders in my API documentation automatically from the spec file?

Yes, the API documentation spec file allows you to create nested folders (second-level subcategories) using the `>` symbol in the tags specified within the OpenAPI spec file (e.g., `Pets > Details`). This enables a well-organized, hierarchical structure of your API endpoints, maintains the hierarchy during resync, and ensures proper display and navigation in the left panel of the API reference.

Can I download articles as PDFs using the API?

Currently, there is no option to download articles as PDFs through the API endpoints.

Helpful links

- [d360 npm package link](#)
- [Source code](#)